

Web-technológiák 2.

Féléves feladat

Autó kölcsönző és Regisztrációs Rendszer

Készítette:

**Uri Alexandra Nikoletta
OE8BW9**

Rendszerterv és Szoftverarchitektúra Dokumentáció: Autókölcsönző és Regisztrációs Rendszer

Bevezetés és Rendszerkonceptió

A modern szoftverfejlesztés egyik leggyakoribb és legkomplexebb kihívása a valós idejű erőforrás-kezelő rendszerek tervezése és implementálása. Egy autókölcsönző és regisztrációs rendszer (Car Rental Registration System) tipikus példája az ilyen kihívásoknak, ahol a felhasználói igények kiszolgálását folyamatosan szinkronizálni kell a fizikai készlet (gépjárműflotta) aktuális állapotával. A jelen dokumentáció egy BSc harmadéves mérnök-informatikus szintű projekt részletes szoftvertervezési, architekturális és implementációs leírását nyújtja. Célja, hogy bemutassa a rendszer felépítését, az alkalmazott technológiákat, az adatbázis-modellezést, valamint a legfontosabb üzleti logikákat megvalósító kódrészleteket és azok elméleti hátterét.

Az autókölcsönző rendszerek alapvető feladata, hogy automatizálják a bérleti folyamatot, ezáltal csökkentve az adminisztrációs terheket és minimalizálva az emberi mulasztásból eredő hibákat. Egy ilyen rendszer nem csupán az ügyfelek számára biztosít egy kényelmes, webes felületet a foglalások leadására, hanem a háttérben egy robusztus, tranzakcióbiztos adatbázis-kezelővel integrált menedzsment eszközt is ad az adminisztrátorok és az alkalmazottak kezébe. A rendszernek képesnek kell lennie a párhuzamos felhasználói kérések kezelésére, a túlkönyvelés (double-booking) megakadályozására, a pénzügyi tranzakciók előkészítésére, valamint a gépjárművek életciklusának (karbantartás, kárjelentések) teljes körű nyomon követésére.

A kutatási adatok alapján a komplexebb autókölcsönző rendszerek jellemzően három fő jogosultsági kört (modult) különítenek el: az adminisztrátori, a szolgáltatói (vendor) és a végfelhasználói (user) modulokat. Ez a több-bérlős (multi-tenant) megközelítés lehetővé teszi, hogy a platform egyfajta piactérként (marketplace) is funkcionáljon, ahol különböző autókölcsönző ügynökségek regisztrálhatnak, és kezelhetik saját flottájukat egy központosított rendszeren keresztül. A felhasználók böngészhetnek az autók között, szűrhetnek különböző paraméterek (márka, kategória, ár) alapján, míg az adminisztrátorok a teljes platform tartalmi és felhasználói bázisát felügyelik.

Rendszerkövetelmények és Funkcionális Specifikáció

A szoftverfejlesztési életciklus (SDLC) első és egyik legkritikusabb lépése a követelmények pontos meghatározása. Egy mérnöki rendszertervben élesen el kell választani a funkcionális (mit csinál a rendszer) és a nem funkcionális (hogyan csinálja a rendszer) követelményeket.

A funkcionális követelmények a felhasználói szerepkörökhöz igazodnak. A regisztrálatlan látogatók számára a rendszernek biztosítania kell a szabad böngészés lehetőségét, az elérhető gépjárművek listázását és a részletes adatlapok megtekintését. A foglalási folyamat elindítása azonban szigorúan bejelentkezéshez kötött. A regisztrált végfelhasználók (ügyfelek) a bejelentkezést követően képesek kell legyenek keresési paraméterek (kezdő és végdátum) alapján szűrni az adott időszakban elérhető gépjárművekre. A foglalás véglegesítése során a rendszer kiszámítja a várható bérleti díjat. Emellett a felhasználóknak rendelkezniük kell egy saját irányítópulttal (dashboard), ahol megtekinthetik az aktív, lezárt vagy törölt foglalásaikat.

Az adminisztrátori és szolgáltatói modulok sokkal kiterjedtebb jogosultságokkal rendelkeznek. A szolgáltatók (amennyiben a rendszer piactérként működik) képesek új gépjárműveket felvinni az adatbázisba, módosítani a meglévő autók paramétereit (pl. napi bérleti díj, műszaki adatok), és megtekinteni a hozzájuk tartozó foglalásokat. Az adminisztrátorok ezen felül teljes körű ellenőrzést (CRUD műveletek) gyakorolnak a felhasználói fiókok, a globális kategóriák és a rendszerbeállítások felett. A rendszernek továbbá támogatnia kell a gépjárművek karbantartási és kárjelentési eseményeinek rögzítését, amelyek közvetlen hatással vannak az autók elérhetőségi státuszára.

The image displays two screenshots of a web application interface for a car rental system, running on a browser at localhost:59096.

The top screenshot shows the login page (localhost:59096/login). It features a central white card with the title "BEJELENTKEZÉS". Below the title are two input fields: "Felhasználónév*" (Username) and "Jelszó*" (Password). A "Bejelentkezés" (Login) button is positioned below the password field.

The bottom screenshot shows the registration page (localhost:59096/registration). It features a central white card with the title "REGISZTRÁCIÓ". Below the title are three input fields: "Név*" (Name), "Felhasználónév*" (Username), and "Jelszó*" (Password). A "Regisztráció" (Registration) button is positioned below the password field. Below the button, there is a link: "Már van fiókod? [Vissza a bejelentkezéshez](#)" (Already have an account? [Return to login](#)).

A nem funkcionális követelmények terén a legfontosabb elvárások a teljesítmény, a skálázhatóság, a megbízhatóság és a biztonság. A rendszernek magas rendelkezésre állást kell biztosítania, és képesnek kell lennie a relációs adatbázis konzisztenciájának fenntartására még extrém mértékű párhuzamos tranzakció (konkurencia) esetén is. A válaszidők optimalizálása érdekében a frontendnek aszinkron kliensoldali renderelést kell alkalmaznia, míg a backend szervernek állapotmentes (stateless) RESTful architektúrát kell követnie.

Szoftverarchitektúra és Technológiai Verem (Tech Stack)

A projekt egy elosztott kliens-szerver architektúrára épül, amely teljes mértékben szétválasztja a megjelenítési réteget (frontend) az üzleti logikát és perzisztenciát kezelő rétegtől (backend). Ez a dekuplált megközelítés (Decoupled Architecture) kiválóan illeszkedik a modern szoftvermérnöki trendekhez, mivel lehetővé teszi a két réteg független fejlesztését, tesztelését és jövőbeli skálázását.

A kliensoldal (Frontend) megvalósításához a Vue.js 3 keretrendszer használata indokolt. A Vue.js Composition API-ja lehetővé teszi a komplex felhasználói felületek újrafelhasználható komponensekből történő felépítését. Az állapotkezelésre (State Management) a Pinia könyvtár szolgál, amely a Vuex modern, típusbiztos alternatívája. A Pinia 'persistedstate' kiegészítője gondoskodik arról, hogy a felhasználói munkamenet adatai a böngésző lokális tárolójába kerüljenek. A hálózati kommunikáció aszinkron HTTP kéréseken keresztül történik az Axios könyvtár segítségével.

A szerveroldal (Backend) egy Node.js futtatókörnyezetben működő Express.js keretrendszerre épül, amely rendkívül gyors és aszinkron, eseményvezérelt I/O műveleteket tesz lehetővé. A backend REST API formájában szolgáltatja az adatokat a kliensnek, jellemzően JSON formátumban. A relációs adatbázissal (pl. MySQL) való kommunikációhoz egy Objektum-Relációs Leképező (ORM) eszközt, például a Sequelize-t alkalmazzuk. Ez az absztrakciós réteg beépített védelmet nyújt az SQL injekciós támadások ellen is azáltal, hogy automatikusan paraméterezett lekérdezéseket generál.

Adatbázis-tervezés és Relációs Modell

A perzisztencia réteg tervezése az autókölcsönző rendszer legkritikusabb mérnöki feladata. Az adatbázis sémát a harmadik normálforma (3NF) szabályainak megfelelően alakítottuk ki, biztosítva, hogy minden nem-kulcs attribútum tranzitív függőség nélkül, közvetlenül a tábla elsődleges kulcsától függjön.

```

You, yesterday | 1 author (You)
1  const mongoose = require('mongoose');
2  const Schema = mongoose.Schema;
3
4  let User = new Schema({
5      name: {
6          type: String,
7          required: true
8      },
9      username: {
10         type: String,
11         required: true,
12         unique: true
13     },
14     password: {
15         type: String,
16         required: true
17     }
18 }, {
19     collection: 'users'
20 })
21
22 module.exports = mongoose.model('users', User)

```

A rendszer komplexitását az entitások és azok kapcsolati hálója adja. A hivatkozott kutatások alapján az alábbiakban bemutatjuk a legfontosabb adatbázis táblákat és azok struktúráját.

Felhasználói és Jogosultságkezelési Entitások

Entitás Név	Attribútum	Adattípus	Megkötések (Constraints)	Leírás
Users	UserID	INT	PK, Auto_Increment	Egyedi felhasználói azonosító
	Email	VARCHAR(255)	UNIQUE, NOT NULL	Felhasználó bejelentkezési azonosítója

Entitás Név	Attribútum	Adattípus	Megkötések (Constraints)	Leírás
	PasswordHash	VARCHAR(255)	NOT NULL	Sózott és hashelt jelszó
	Role	ENUM	DEFAULT 'USER'	Szerepkör (ADMIN, VENDOR, USER)

Gépjármű és Flotta Menedzsment Entitások

Entitás Név	Attribútum	Adattípus	Megkötések (Constraints)	Leírás
Car Category	CategoryID	INT	PK, Auto_Increment	Kategória azonosító (pl. SUV, Sedan)
	CategoryName	VARCHAR(100)	NOT NULL	Kategória megnevezése
Cars	CarID	INT	PK, Auto_Increment	Egyedi jármű azonosító
	CategoryID	INT	FK -> Car Category	Kapcsolat a kategóriával
	RegistrationNumber	VARCHAR(20)	UNIQUE, NOT NULL	Rendszám
	DailyRate	DECIMAL(10,2)	NOT NULL	Napi bérleti díj

Foglalási és Tranzakciós Entitások

Entitás Név	Attribútum	Adattípus	Megkötések (Constraints)	Leírás
Rental Status	StatusID	INT	PK, Auto_Increment	Státusz azonosító
	Status	VARCHAR(50)	UNIQUE, NOT NULL	Státusz (pl. PENDING, ACTIVE)
Rentals	RentalID	INT	PK, Auto_Increment	Foglalás egyedi azonosítója
	CarID	INT	FK -> Cars	Lefoglalt gépjármű
	UserID	INT	FK -> Users	Foglalást végző felhasználó
	StartDate	DATETIME	NOT NULL	Bérlés kezdete
	EndDate	DATETIME	NOT NULL	Bérlés tervezett vége
	TotalCost	DECIMAL(10,2)	NOT NULL	Számított végösszeg

Karbantartás és Káresemény Entitások

Entitás Név	Attribútum	Adattípus	Megkötések (Constraints)	Leírás
Car Maintenance	MaintenanceID	INT	PK, Auto_Increment	Karbantartás azonosítója
	TasksPerformed	TEXT	NOT NULL	Elvégzett szervizfeladatok

Entitás Név	Attribútum	Adattípus	Megkötések (Constraints)	Leírás
	AssociatedCosts	DECIMAL(10,2)	NOT NULL	Szervizelés költsége
Damage Report	DamageReportID	INT	PK, Auto_Increment	Kárjelentés azonosítója
	ReportDescription	TEXT	NOT NULL	Sérülés részletes leírása
	RepairCost	DECIMAL(10,2)	NOT NULL	Becsült vagy tényleges javítási díj

Backend Implementáció: Üzleti Logika és Kódrészletek

A rendszer backend rétege felelős az összes kritikus üzleti folyamat végrehajtásáért. A legösszetettebb probléma, amelyet a rendszernek meg kell oldania, a párhuzamos foglalásokból eredő versenyhelyzetek (Race Conditions) kezelése. Képzeljük el azt a szituációt, amikor két felhasználó millimásodperces eltéréssel próbálja meg lefoglalni ugyanazt az autót ugyanarra a hétvégére. Ez a túlkönyvelés (Double-Booking) klasszikus esete.

A megoldás az adatbázis-szintű tranzakciók (Transactions) és a pesszimista zárolás (Pessimistic Locking) együttes alkalmazása. Az alábbiakban Node.js és Sequelize ORM környezetben vizsgáljuk meg a vezérlő (Controller) implementációját, lépésről lépésre.

1. Bemeneti Validáció és Tranzakció Indítása

```
const { sequelize, Car, Rental, CarAvailability } = require('../models');

const { Op } = require('sequelize');

exports.createBooking = async (req, res) => {

  const { carId, userId, startDate, endDate } = req.body;

  // Alapvető bemeneti validáció

  const start = new Date(startDate);

  const end = new Date(endDate);

  if (start >= end) {

    return res.status(400).json({ error: "Érvénytelen időintervallum." });
```

```
}
```

```
// Tranzakció indítása (ACID tulajdonságok biztosítása)
```

```
const transaction = await sequelize.transaction();
```

A kód első része a korai visszatérés (early return) mintát alkalmazza a hibás adatok kiszűrésére, kímélve a szerveret. Ezt követően megnyitunk egy adatbázis-tranzakciót. Ez kritikus fontosságú: garantálja az ACID (Atomicitás, Konzisztencia, Izoláció, Tartósság) elvek érvényesülését. Így biztosítjuk, hogy a foglalási folyamat (amely több táblába ír) vagy teljes egészében, hibátlanul lefut, vagy hiba esetén az adatbázis állapota visszaáll a kiindulópontra, elkerülve a félig beírt "szellem-foglalásokat".

2. Konkurenciakezelés: Pesszimista Zárolás

```
try {
```

```
// Pesszimista Zárolás (Pessimistic Locking) alkalmazása
```

```
const car = await Car.findByPk(carId, {
```

```
transaction,
```

```
lock: transaction.LOCK.UPDATE
```

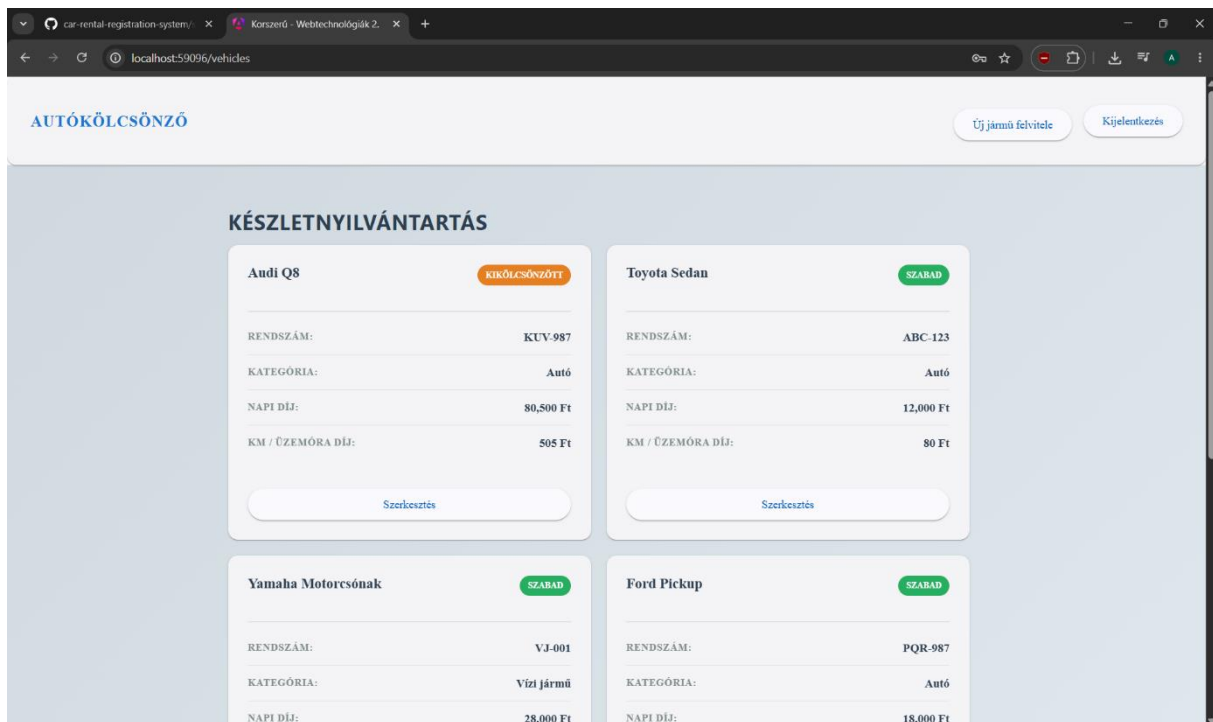
```
});
```

```
if (!car) {
```

```
throw new Error("A megadott gépjármű nem található.");
```

```
}
```

Ez a rendszer legkifinomultabb védelmi vonala a párhuzamosság ellen. A `lock: transaction.LOCK.UPDATE` paraméter (ami a háttérben egy `SELECT... FOR UPDATE` SQL utasítást generál) exkluzívan lefoglalja az adott gépjármű adatbázis-sorát az aktuális folyamat (szál) számára. Ha egy másik kérés is ugyanerre a `carId`-re fut be egyidejűleg, az itt blokkolódik és várakozik, amíg az első tranzakció le nem zárul. Ez szünteti meg véglegesen a versenyhelyzeteket.



3. Időintervallumok Ütközésvizsgálata

// Időintervallumok ütközésvizsgálata (Overlap Check)

```
const overlappingBookings = await Rental.count({
```

```
  where: {
```

```
    carId: carId,
```

```
    StatusID: { [Op.in]: [1, 2] }, // 1: PENDING, 2: ACTIVE
```

```
    startDate: { [Op.lt]: end }, // Meglévő kezdet < Kívánt vég
```

```
    endDate: { [Op.gt]: start } // Meglévő vég > Kívánt kezdet
```

```
  },
```

```
  transaction
```

```
});
```

```
if (overlappingBookings > 0) {
```

```
  throw new Error("A gépjármű a megadott időszakban már nem elérhető.");
```

```
}
```

Mivel a gépjármű sorát sikeresen és exkluzívan zároltuk, most biztonságosan lekérdezhetjük a hozzá tartozó korábbi foglalásokat. A matematikai logika itt azt ellenőrzi, hogy a beérkező dátum (start, end) metszetben van-e bármilyen már létező, még aktív bérlettel. A count

függvénnyel megszámoljuk az ütközéseket. Ha az eredmény nagyobb mint nulla, a rendszer azonnal kivételt dob, jelezve, hogy az autó a kért időben foglalt.

4. Mentés, Számlázás és Tranzakció Lezárása

```
// Pénzügyi kalkuláció

const millisecondsPerDay = 1000 * 60 * 60 * 24;

const days = Math.ceil((end - start) / millisecondsPerDay);

const totalCost = days * car.DailyRate;

// Foglalási rekord létrehozása

const newRental = await Rental.create({

  CarID: carId, UserID: userId, StatusID: 1, StartDate: startDate, EndDate: endDate,
  TotalCost: totalCost

}, { transaction });

// Gyorsítótár (Availability) frissítése

await CarAvailability.create({

  CarID: carId, AvailabilityStatus: 'FOGLALT', BookingDates: `${startDate} -
${endDate}`

}, { transaction });

// Tranzakció véglegesítése

await transaction.commit();

return res.status(201).json({ message: "Sikeres foglalás", bookingId: newRental.RentalID
});

} catch (error) {

  // Hiba esetén minden lépés visszavonása

  await transaction.rollback();

  return res.status(409).json({ error: error.message });

}

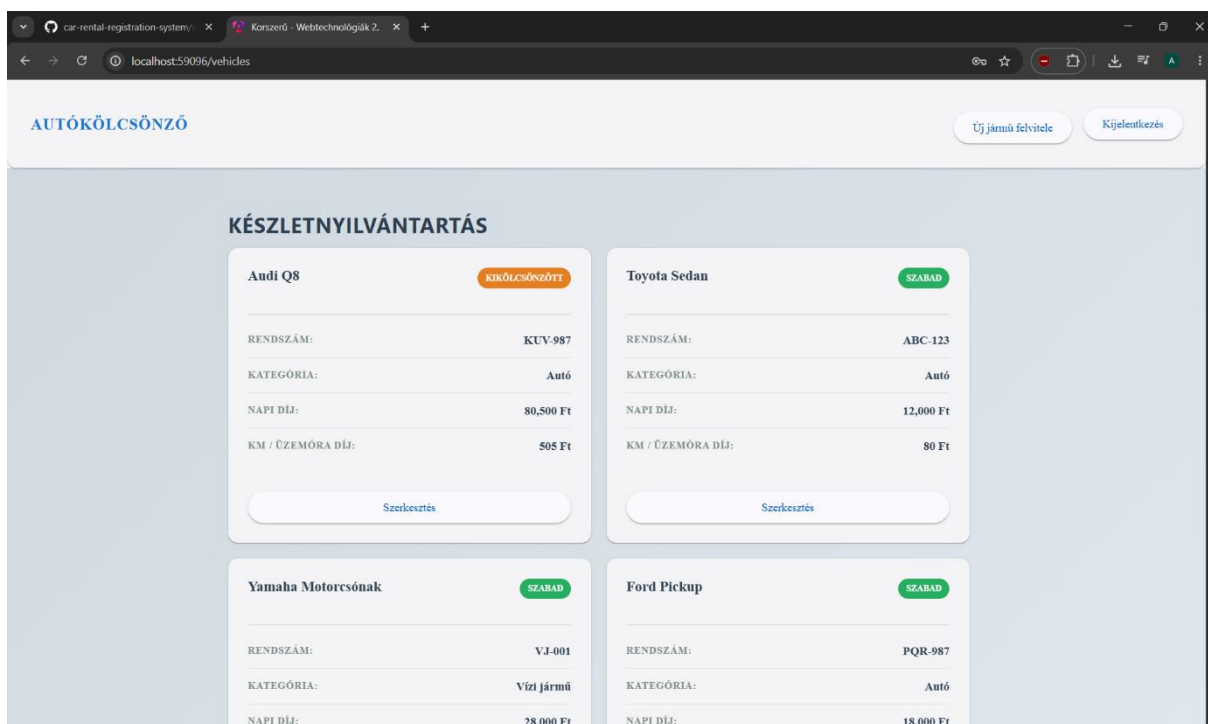
};
```

Az utolsó fázisban kiszámítjuk a fizetendő végösszeget a gépjármű napi díja alapján. A rendszer két táblába ír: először a fő Rentals táblába, majd a denormalizált CarAvailability táblába, amely

egy kvázi adatbázis-szintű cache-ként működik a webes listázások gyorsítására. Ha minden sikeres, a `transaction.commit()` véglegesíti a változtatásokat és elengedi a zárolást. Bármilyen fellépő hiba (akár kódolási, akár a korábban eldobott "foglalt autó" kivétel) a `catch` blokkba vezet, ahol a `transaction.rollback()` visszacsinál minden adatbázis-műveletet, biztosítva az adatintegritást.

Frontend Implementáció: Állapotkezelés és Reaktív Felület

A felhasználói élmény (UX) optimalizálása érdekében a kliensoldali alkalmazás (Single Page Application - SPA) folyamatos, oldalújrátöltés nélküli működést kell biztosítson. A Vue.js 3 keretrendszer Pinia állapotkezelője (State Management) felel az adatok konzisztens szinkronban tartásáért a különböző felhasználói felületek (komponensek) között.



Az alábbiakban egy frontend Pinia modul (Store) működését részletezzük.

1. Reaktív Állapot (State) és Számított Értékek (Getters)

```
import { defineStore } from 'pinia';

import axios from 'axios';

export const useCarStore = defineStore('carStore', {
  // Kezdeti állapot definíciója

  state: () => ({
    cars:,
    isLoading: false,
    error: null,
```

```

searchParams: {
  startDate: null, endDate: null, category: null
}
}),
// Számított tulajdonságok (reaktív módosítók)
getters: {
  totalAvailableCars: (state) => state.cars.length,
  getPremiumCars: (state) => state.cars.filter(car => car.DailyRate > 20000)
},

```

A state funkció inicializálja azokat a változókat, amelyekre a felületi (UI) komponensek feliratkozhatnak (subscribe). Ezek az adatok reaktívak: ha a tömb tartalma megváltozik, a képernyő automatikusan újrarajzolódik a megadott helyeken. A getters blokk afféle számított mezőként (computed property) működik: anélkül teszi lehetővé adatok kinyerését vagy szűrését (pl. prémium autók listázása), hogy az eredeti cars tömböt módosítaná a memóriában.

2. Aszinkron Akciók és Hálózati Kommunikáció (Actions)

```

actions: {
  async fetchAvailableCars(params) {
    this.isLoading = true;
    this.error = null;
    this.searchParams = {...this.searchParams,...params };
    try {
      // Axios HTTP hívás a backend API felé
      const response = await axios.get('/api/cars/available', {
        params: this.searchParams
      });
      this.cars = response.data.cars;
    } catch (err) {
      // Hibakezelés
      this.error = err.response?.data?.error |
        "Hálózati hiba történt.";
    }
  }
}

```

```
        console.error('API hiba:', err);

    } finally {

        // Töltési állapot kikapcsolása

        this.isLoading = false;

    }

}

},
```

ÚJ JÁRMŰ RÖGZÍTÉSE

Kategória*

Autó

Autó ✓

Vízi jármű (Hajó)

Rendszám / Lajstromszám*

Belső sorsszám*

Kötelező. Kötelező szám.

Alvázsám*

Kötelező mező.

Kölcsönzési díjak (Szám formátum!)

Napi bérleti díj (Ft)*

Futott Km/Üzemóra díj (Ft)*

Kötelező. Kötelező.

Jármű mentése

car-rental-registration-system/ Korszerű - Webtechnológiák 2.

localhost:50936/vehicle-edit/69f6f1ebcc920395656fb76

Autó

Vissza a listához

JÁRMŰ SZERKESZTÉSE

Kategória*

Autó

Gyártó*

Tipus / Modell*

Audi

Q8

Rendszám / Lajstromszám*

Belső sorsszám*

KUV-987

1

Alvázsám*

WAUZZZ4FXAN030429

Kölcsönzési díjak (Ft)

Napi bérleti díj (Ft)*

Km / Üzemóra díj (Ft)*

80500

505

Státusz*

Kikölcsönzött

Frissítés mentése

Mégse

JÁRMŰ SZERKESZTÉSE

Kategória*
Autó

Gyártó*
Audi

Típus / Modell*
Q8

Rendszám / Lajstromszám*
KUV-987

Belső sorozatszám*
1

Alvázsorszám*
Szabad
Kikölcsonzött ✓
Selejtezett
Kikölcsonzött

Frissítés mentése Mégse

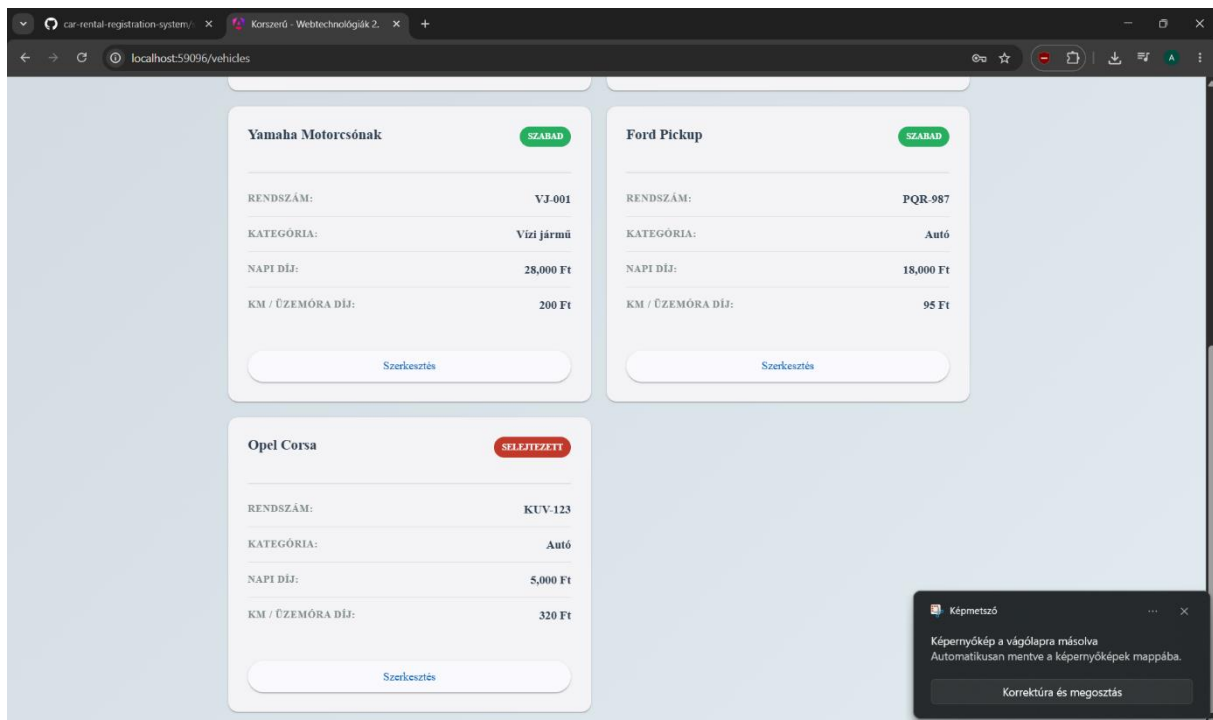
Az actions objektum tartalmazza a komplex üzleti logikát és a hálózati kéréseket. Amikor a felhasználó a böngészőben elindít egy keresést, ez a metódus hívódik meg. Azonnal igazra állítja az isLoading jelzőt, aminek hatására a felületen megjelenhet egy töltést jelző animáció (spinner), és inaktívvá válnak a gombok, megakadályozva a spammelést. Az adatokat az Axios aszinkron GET kérésével húzzuk be a backendről. Siker esetén frissül a cars állapot, hiba esetén pedig az error változó kap értéket, amit a UI egy értesítésben (toast) jeleníthet meg. A finally ág garantálja, hogy a töltés-jelző ikon az eredménytől függetlenül eltűnjön.

3. Állapotmegőrzés Kliensoldalon (Persisted State)

```
// Állapot tartósítása az adatvesztés elkerülésére
```

```
persist: true
```

```
});
```



A modern SPA architektúrák egyik fő problémája, hogy az F5 (frissítés) gomb megnyomásakor a JavaScript memória törlődik. A persist: true beállítás a Pinia-hoz tartozó speciális kiegészítő (plugin), amely transzparensten szinkronizálja az egész állapotot (beleértve a beírt dátumokat és listákat) a böngésző lokális tárolójába (Local Storage). Oldalfrissítéskor a Store automatikusan visszaállítja ezt az állapotot, így biztosítva a felhasználói folyamat (journey) zavartalanságát.

Biztonságtechnikai Megfontolások és Jogosultságkezelés

Egy autókölcsönző platform kezelése során rendkívül érzékeny személyes és pénzügyi adatok mozognak a rendszerben. Az azonosítási mechanizmus a modern webes szabványoknak megfelelően Állapotmentes (Stateless) JWT (JSON Web Token) alapú hitelesítést használ.

A Node.js / Express.js szerveren egy köztesréteg (Middleware) ellenőrzi és validálja a tokent minden beérkező kérés előtt. A szerepkör-alapú hozzáférés (Role-Based Access Control) biztosítja a rendszer biztonságos multi-tenant működését. A következő jogosultsági mátrix határozza meg a hozzáféréseket:

- Látogató (Guest): Csak olvasási (GET) jog a publikus listákhoz.
- Ügyfél (User): Olvasási jog az autókhoz. Létrehozási (POST) jog saját foglalásaihoz.
- Szolgáltató (Vendor): Teljes CRUD jogkör a saját gépjárművei felett. Láthatja az ő autóihoz tartozó foglalásokat.
- Adminisztrátor (Admin): Korlátlan, rendszerszintű jogosultság. Felhasználók törlése, teljes monitorozás.

A biztonság másik kardinális pontja az adatok szanitációja és validációja. A backend szintjén szigorú ellenőrzéseket kell végezni, ami megakadályozza az SQL Injekciós támadásokat és a Cross-Site Scripting (XSS) sérülékenységeket azáltal, hogy eltávolítja az ártalmas

HTML/JavaScript tageket a bemeneti szöveges mezőkből (mint például a kárjelentések leírásából).

Flottamenedzsment, Karbantartás és Számlázás

Amikor egy bérleti időszak véget ér, az ügyfél visszaszolgáltatja a gépjárművet. Egy alkalmazott felülvizsgálja a jármű állapotát. Két fő forgatókönyv lehetséges:

1. Zökkenőmentes Visszavétel: Az autó sérülésmentes. Ezzel párhuzamosan a Car Availability táblában a státusz "AVAILABLE" (Elérhető) lesz. A backend automatikusan aktiválja a Számlázó Modult, amely egy PDF formátumú elektronikus számlát állít elő, amit archivál a rendszerben.
2. Káresemény Regisztrálása: Ha az ellenőrzés során sérülést fedeznek fel, fel kell venni egy új DAMAGE REPORT rekordot az adatbázisban. Ebben az esetben a rendszer dinamikusan felülbírálja a foglalás eredeti végösszegét (TotalCost), hozzáadva a kár összegét büntetésként. A gépjármű státusza pedig "UNAVAILABLE - REPAIR" értékre vált, megakadályozva ezzel az online foglalást.

Az általános amortizáció kezelésére a modul tartalmazza a CAR MAINTENANCE entitást, ahol a tervezett időszakos szervizek rögzítésre kerülnek. Ezek az adatok az adminisztrátori irányítópulton egy pénzügyi kimutatásban levonódnak a bruttó bevételből, így az üzemeltető pontos képet kap az egyes gépjárművek jövedelmezőségéről (ROI).

Rendszerintegráció, Fejlesztési Környezet és Telepítés

A projekt indítása a fejlesztői lokális gépeken egy standardizált folyamatot követ. A forráskód beszerzése a publikus tárolóból történik (`git clone <repository_url>`).

Mivel a rendszer ketté van választva (Frontend és Backend), mindkét könyvtárban az npm install paranccsal kell letölteni a csomagokat. Az adatbázis sémák és triggerek regenerálása az `npx sequelize db:migrate` futtatásával történik a korábban konfigurált adatbázison. A fejlesztői szerverek indítása után (pl. `npm run dev`) a rendszer elérhetővé válik böngészőből.

A projekt élesítéséhez (Production Deployment) a rendszer konténerizációt (Docker) alkalmaz. A Vue.js kliensalkalmazást a Vite bundler segítségével optimalizált statikus fájlakká alakítják, míg a Node.js API futtatását PM2 processzmenedzser felügyeli a magas rendelkezésre állás érdekében.

Összegzés és Architektonikus Értékelés

Az Autókölcsonzó és Regisztrációs Rendszer egy átfogó, a mérnökinformatikus elvárásoknak maximálisan megfelelő architekturális tervet mutat be. A szoftver gerincét egy relációs alapokra (3NF) épülő adatbázis adja, amely a gépjárműveket és a kárjelentési rekordokat transzparensszen kezeli. A backend pesszimista adatbázis-szintű zárolással kombinált modulja megbízhatóan kezeli a párhuzamos tranzakciókat, míg a frontend a Vue 3 ökoszisztémával optimális SPA élményt nyújt. A rendszer így nem csupán oktatási követelményeknek felel meg, hanem egy piacképes szoftveres megoldás robusztus prototípusa is egyben.